

Panduan Belajar: Tech Stack Proyek

Tugas Besar Big Data · Credit-Card Default (Taiwan 2005)

Penjelasan tiap teknologi — apa, kenapa dipakai di sini, konsep kunci, perannya di proyek, cara belajar.

Tujuan dokumen: memahami **mengapa** setiap teknologi dipilih dan **bagaimana** mereka bekerja sama, bukan sekadar daftar tools. Dikaitkan dengan file nyata di repo.

0. Gambaran besar — bagaimana semuanya tersambung

```
SUMBER DATA EXTRACT TRANSFORM WAREHOUSE DASHBOARD UCI (XLSX/CSV) ■ ■ [ETL] PySpark ■ ■■> Python ■> Apache
Kafka ■■■■■>■ ■■> Apache Hive ■> Tableau (ETL) FRED API(JSON) ■ (requests) (KRaft) ■ [ELT] Hive SQL■ (+
PostgreSQL) Power BI (ELT) (file Parquet) CTGAN (SDV) memperbanyak data kredit jadi 150.000 sebelum masuk
pipeline Semua dibungkus & dijalankan dengan DOCKER COMPOSE
```

Inti alur: data diambil (Python) → dialirkan (Kafka) → diolah (Spark untuk ETL / SQL Hive untuk ELT) → disimpan di warehouse (Hive, format Parquet, metadata di PostgreSQL) → divisualkan (Tableau & Power BI). Semua komponen jalan sebagai container yang dirangkai **Docker Compose**.

1. Python — bahasa perekat (glue language)

- **Apa:** bahasa pemrograman utama proyek.
- **Kenapa di sini:** ekosistem data terkaya — punya library untuk Spark (PySpark), data (pandas), API (requests/fredapi), dan AI sintetis (SDV/CTGAN). Satu bahasa untuk semua tahap.
- **Konsep kunci:** virtual env, pip + requirements.txt (pin versi agar reproduisibel), tipe data, fungsi & docstring.
- **Peran di proyek:** semua script — `etl_pipeline/*.py`, `elt_pipeline/*.py`, `synthetic/train_ctgan.py`, `dashboard/build_dashboard_kpis.py`. Dependensi terkunci di `requirements.txt`.
- **Cara belajar:** dasar Python -> pandas (DataFrame) -> baru PySpark.

2. Apache Kafka (mode KRaft) — sistem antrian pesan (message bus)

- **Apa:** platform *streaming* pesan; pengirim (**producer**) menaruh pesan ke **topic**, penerima (**consumer**) mengambilnya. Tahan-banting & teratur.
- **Kenapa di sini:** memisahkan tahap *extract* dari pemrosesan. Sumber data dipublikasikan ke topic, lalu consumer menyimpannya ke `raw/` & `datalake/`. Membuat extract bisa di-*stream* & idempoten (diulang tanpa rusak). Memenuhi syarat "Extract = Kafka" pada brief.
- **Konsep kunci:** broker, topic, partition, producer/consumer, offset. **KRaft** = mode Kafka modern tanpa Zookeeper (lebih simpel).
- **Peran di proyek:** `etl_pipeline/kafka_producer.py` & `kafka_consumer.py`; konfigurasi broker di `docker-compose.yml` (service kafka, listener INTERNAL kafka:19092 & EXTERNAL 9092).
- **Cara belajar:** pahami analogi "kantor pos" (producer kirim surat -> topic kotak -> consumer ambil); coba buat 1 topic + kirim/baca pesan (lihat smoke test di README).

3. Apache Spark / PySpark — mesin pemrosesan data terdistribusi

- **Apa:** engine komputasi cepat untuk data besar; API Python-nya = **PySpark**.
- **Kenapa di sini:** menangani transform **baris-per-baris** berskala besar (900.000 baris) dengan cepat — cleaning, unpivot, feature engineering. Inti dari jalur **ETL**.

- **Konsep kunci:** **DataFrame** (tabel terdistribusi), *lazy evaluation* (operasi baru jalan saat ada *action* seperti `count/write`), `local[*]` (jalan di semua core 1 mesin) vs cluster (master+worker), transformasi (`select`, `join`, `groupBy`, `withColumn`).
- **Peran di proyek:** `etl_pipeline/transform.py` (clean, IQR/Z-score, unpivot ke bulanan, 5 fitur), `load.py` (tuliskan ke Hive). Sesi Spark dibuat di `common/spark_session.py`. Image: `docker/Dockerfile.spark` (basis `apache/spark:3.5.3-python3`).
- **Cara belajar:** pandas dulu (konsep DataFrame mirip), lalu PySpark + konsep lazy/partition.

4. Apache Hive — data warehouse berbasis SQL

- **Apa:** lapisan yang membuat data di file (Parquet) bisa di-query pakai **SQL**, lengkap dengan katalog tabel. Terdiri dari **Metastore** (penyimpan metadata/skema) + **HiveServer2** (endpoint JDBC/ODBC tempat tool BI menyambung).
- **Kenapa di sini:** menjadi **gudang data** tunggal yang bisa diakses Tableau & Power BI lewat ODBC, dan tempat transform **ELT** dijalankan (SQL window/OLAP).
- **Konsep kunci:** metastore vs server, tabel managed, SQL (`CREATE TABLE AS`, window function `LAG/RANK`, `GROUPING SETS` OLAP), warehouse directory.
- **Peran di proyek:** warehouse `bigdata_etl` (star schema) & `bigdata_elt` (tabel analitik). DDL: `warehouse/etl_star_schema.sql`; query: `warehouse/etl_analytical_queries.sql`; transform ELT: `etl_pipeline/transform.sql`. Image: `docker/Dockerfile.hive`.
- **Cara belajar:** SQL dasar -> konsep data warehouse (fakta vs dimensi) -> window function & OLAP.

5. PostgreSQL — database pendukung metastore Hive

- **Apa:** database relasional (RDBMS) yang andal.
- **Kenapa di sini:** Hive Metastore butuh database untuk menyimpan **metadata** (daftar tabel, kolom, lokasi file). Postgres dipilih sebagai *backing store*-nya.
- **Konsep kunci:** RDBMS, tabel, driver JDBC (Hive butuh `postgresql-42.7.4.jar` untuk konek).
- **Peran di proyek:** service postgres di `docker-compose.yml` (DB metastore). **Tidak** diekspos ke publik (aman). Driver disalin di `docker/Dockerfile.hive`.
- **Cara belajar:** cukup pahami perannya sebagai "buku katalog" Hive; tak perlu mendalam untuk proyek ini.

6. Apache Parquet — format penyimpanan kolumnar

- **Apa:** format file biner **kolumnar** (menyimpan per-kolom, bukan per-baris) + terkompresi.
- **Kenapa di sini:** jauh lebih cepat & hemat untuk query analitik dibanding CSV; bisa dibaca Hive & ODBC. Semua tabel warehouse `STORED AS PARQUET / USING PARQUET`.
- **Konsep kunci:** kolumnar vs baris, kompresi (snappy), schema melekat di file, *splittable*.
- **Peran di proyek:** output staging `warehouse/staging/*.parquet`; tabel Hive & KPI pakai Parquet agar terbaca ODBC.
- **Cara belajar:** bandingkan ukuran & kecepatan CSV vs Parquet untuk data yang sama.

7. CTGAN (via SDV) — pembangkit data sintetis

- **Apa:** **Conditional Tabular GAN** — model AI yang belajar pola data tabel lalu menghasilkan baris baru yang mirip. Disediakan library **SDV** (CTGANSynthesizer).
- **Kenapa di sini:** data UCI hanya ~30.000 baris; brief minta ≥ 100.000 . CTGAN memperbanyak jadi **150.000 nasabah** tanpa mengumpulkan PII baru, dan *conditional sampling* menjaga rasio default ~22%.
- **Konsep kunci:** GAN (generator vs discriminator), data mixed-type, conditional sampling, seed (reproduksibilitas), **bias/limitasi** (fidelitas $\leq$ sumber — wajib didokumentasikan).
- **Peran di proyek:** `synthetic/train_ctgan.py`; metode & bias di `synthetic/CTGAN_METHOD.md`; output `synthetic/synthetic_credit_clients.csv` (= sumber kredit 1). Dijalankan **sekali** (berat, pakai PyTorch) lalu di-skip.
- **Cara belajar:** konsep GAN tingkat tinggi -> SDV quickstart -> baca `CTGAN_METHOD.md`.

8. FRED API + UCI Repository — sumber data

- **Apa:** **UCI** = file dataset kartu kredit (XLSX/CSV); **FRED** = API ekonomi (JSON) dari Federal Reserve.
- **Kenapa di sini:** memenuhi syarat **multi-source** (2 sumber) & **multi-format** (XLSX/CSV + JSON, bonus). UCI = data perilaku nasabah; FRED = konteks makro (kurs, cadangan devisa Taiwan 2005).
- **Konsep kunci:** REST API, parameter query, JSON, API key (rahasia -> `.env`), rate limit.
- **Peran di proyek:** `etl_pipeline/extract.py` (`extract_etl_source1` UCI, `extract_etl_source2` FRED API). Kunci di `.env` (`FRED_API_KEY`).
- **Cara belajar:** pahami cara kerja REST API + JSON; lihat dokumentasi FRED API.

9. Docker & Docker Compose — pengemasan & orkestrasi

- **Apa:** **Docker** mengemas aplikasi + dependensinya jadi **container** yang jalan sama di mana saja. **Docker Compose** menjalankan **banyak** container sekaligus dari satu file.
- **Kenapa di sini:** stack ini punya 7 layanan rumit (Kafka, Spark x2, Hive x2, Postgres, UI). Compose membuatnya **reproduktibel sekali jalan** (`docker compose up`) di laptop maupun cloud.
- **Konsep kunci:** image vs container, Dockerfile (resep image), volume (data permanen), network (container saling memanggil via nama service), port mapping, multi-arch (amd64 vs ARM).
- **Peran di proyek:** `docker-compose.yml` (mode aman, port `127.0.0.1`), `docker-compose.prod.yml` (override ekspos publik/insecure), `docker/Dockerfile.spark` & `Dockerfile.hive`.
- **Cara belajar:** jalankan 1 container sederhana -> pahami volume & port -> baca `docker-compose.yml` proyek baris demi baris.

10. Tableau & Power BI — visualisasi (dashboard)

- **Apa:** tool BI untuk membuat dashboard interaktif dari data.
- **Kenapa di sini:** brief minta dashboard yang sama di **dua** tool: **Tableau** -> **output ETL**, **Power BI** -> **output ELT**, agar dua pipeline bisa dibandingkan secara visual.
- **Konsep kunci:** koneksi **ODBC** (jembatan ke HiveServer2), measure vs dimension, filter interaktif, KPI card, dual-axis, scatter.
- **Peran di proyek:** baca tabel `kpi_*` dari Hive. Panduan: `dashboard/DASHBOARD_BUILD_GUIDE.md`, koneksi: `dashboard/HIVE_ODBC_CONNECTION.md`, fallback CSV: `dashboard/export_kpis_csv.py`.
- **Cara belajar:** ikuti tutorial dasar masing-masing tool -> hubungkan ke CSV `kpi_*` dulu (paling mudah) -> baru ODBC.

11. Konsep besar: ETL vs ELT (kenapa proyek pakai keduanya)

- **ETL** (Extract-Transform-Load): olah data **sebelum** masuk warehouse. Di sini = **PySpark** (baris-per-baris, fitur ML, validasi) -> star schema bersih. Cocok untuk pemodelan & tata kelola.
- **ELT** (Extract-Load-Transform): muat data mentah dulu, olah **di dalam** warehouse pakai **SQL**. Di sini = **Hive SQL** (window function, OLAP). Cocok untuk *landing* cepat & eksplorasi.
- **Temuan terukur proyek:** ELT ~5x lebih cepat (set-based SQL) vs ETL (compute Python di hulu); ETL unggul di kualitas fitur. Detail: `analysis/ETL_vs_ELt_comparison.md`.

12. Glosarium singkat

Istilah	Arti singkat
Broker	server Kafka yang menyimpan pesan
DataFrame	tabel data di pandas/Spark

Lazy evaluation	komputasi ditunda sampai diperlukan (Spark)
Metastore	katalog metadata tabel Hive
HiveServer2	endpoint JDBC/ODBC Hive
ODBC	standar koneksi database untuk tool BI
Star schema	desain warehouse: 1 tabel fakta + beberapa dimensi
Window function	SQL yang menghitung antar-baris (LAG, RANK)
Parquet	format file kolumnar terkompresi
GAN	model AI generator vs discriminator
Container	aplikasi terkemas yang jalan terisolasi
Multi-arch	image Docker untuk amd64 & ARM

13. Saran urutan belajar (dari fondasi ke atas)

1. **Python + pandas** (fondasi semua).
2. **SQL** (untuk Hive/ELT & warehouse).
3. **Konsep Docker** (image/container/volume) -> baca `docker-compose.yml`.
4. **PySpark** (DataFrame, lazy, transformasi) -> baca `etl_pipeline/transform.py`.
5. **Hive & data warehouse** (star schema, window/OLAP) -> baca `warehouse/*.sql`.
6. **Kafka** (producer/consumer/topic) -> smoke test.
7. **CTGAN/SDV** (data sintetis + bias) -> `synthetic/CTGAN_METHOD.md`.
8. **Tableau/Power BI** (dashboard via ODBC/CSV).
9. **Docker Compose end-to-end + hosting** -> `docs/HOSTING_ORACLE.md`.

14. Rujukan dokumen lain di repo

- `docs/DOKUMENTASI LENGKAP.pdf` — referensi komponen per-file.
- `docs/PROSES_ETL_ELТ.pdf` — alur data tahap demi tahap.
- `docs/CARA_MENJALANKAN.pdf` — cara menjalankan pipeline.
- `docs/HOSTING_ORACLE.pdf` — hosting ke cloud.
- `docs/PANDUAN_BELAJAR_GITHUB_HOSTING.pdf` — belajar Git/GitHub & hosting.
- `report/report.md` (`report.pdf`) — laporan akademik lengkap.